

Drought Severity Prediction – Ergebnisse & Analyse

Jana Speldrich

2026-06-08

Table of contents

Überblick	2
Ergebnisse	2
Kaggle-Scores im Überblick	2
Implementierungsunterschiede	4
Feature Importance (LightGBM Gain)	4
v19 und v20 — kein score_lag (aktuellste Runs)	4
v21a — Feature-Shift durch längere Rolling-Fenster (270d + 365d)	5
v22_recent8 — Feature-Shift durch Recent-Filter	6
Ältere Runs — mit score_lag	6
Analyse: Warum score_lag schadet	7
Das Staleness-Problem	7
v18: Gap-Simulation — Ergebnisse & Interpretation	7
v22_recent8: Recent-Filter — Ergebnisse & Interpretation	8
RECENT_YEARS Sweep — Vollständige Ergebnisse	9
v24: 8 Jahre + Seasonal Mean — Ergebnisse & Interpretation	11
v26: RECENT_YEARS=7 — Überraschender Ausreißer	12
v21a: Längere Rolling-Fenster (270d + 365d) — Ergebnisse & Interpretation	13
v21b: Mehr Bäume (LR=0.02, N=2000, $\times 1.1$) — Ergebnisse & Interpretation	15
v25: Last-Year Validation (52 Fenster/Region) — Ergebnisse & Interpretation	16
Validierungsschema – Analyse & Alternativen	17
Aktuelles Schema	17
Unbekannte Teststruktur	17
Alternative Validierungsstrategien	18
Option 1: Time-based Split (empfohlen als Vergleich)	18

Option 2: Gap-Simulation (realistischstes Val-Schema)	18
Option 3: Gemischte Strategie	18
Empfehlung: Was testen?	19

Nächste Schritte	19
-------------------------	-----------

Technische Details	20
Datenstruktur NPZ	20
Cache-Strategie	20
Modell-Parameter (alle Scripts)	20

Überblick

Kaggle-Wettbewerb zur Vorhersage von Dürreschwere (Score 0–5) für **2.248 Regionen**, jeweils **5 Wochen** in die Zukunft. Metrik: **MAE** (Mean Absolute Error), niedriger = besser.

- Trainingsdaten: täglich, ~13 Jahre pro Region (Ordinal-Jahre 1000–8133)
- Testdaten: 91 Tage pro Region (kein Score vorhanden)
- Limit: 3 Submissions pro Tag

Ergebnisse

Kaggle-Scores im Überblick

Script	Val-Schema	RECENT_YEARS	Val MAE	Kaggle MAE	Δ vs. v12	Bemerkung
recent_local (<i>CSV, holdout</i>)	20%-Holdout	8	0.2661	0.8095	−0.0163	Bestes Modell
v24 (8y + seasonal_mean)	Last-window all	8	0.2314	0.8106	−0.0152	2. bestes!
v22 (<i>Kaggle NB3, last-window</i>)	Last-window all	5	0.2342	0.8132	−0.0126	
v25 (Last-year val, 52w/region)	Last-year (52 Win.)	9	—	0.8163	−0.0095	

Script	Val-Schema	RECENT_YEARS	Val MAE	Kaggle MAE	Δ vs. v12	Bemerkung
v21b (LR=0.02, N=2000, $\times 1.1$)	Last-window all	15	0.2528	0.8192	-0.0066	v19
v19 (<i>Last-window all regions</i>)	Last-window all	15	0.2522	0.8193	-0.0065	
v12 (<i>Baseline</i>)	20%-Holdout	15	~0.04	0.8258	—	
v26 (7y, last-window)	Last-window all	7	0.2273	0.8270	+0.0012	7y = Loch!
v21a (ROLL_WINS +270d+365d)	Last-window all	15	0.2272	0.8671	+0.041	91d-Grenze!
recent20_local (<i>CSV, holdout</i>)	20%-Holdout	5	0.2323	0.8654	+0.040	5y zu wenig
v20 (Schema A, K=5 Val-Fenster)	Schema A	15	0.2814	0.8728	+0.047	
v18 (gap=13 score_lag)	Last-window all	15	0.1831	0.9134	+0.088	
v16_chain (autoregressive)	—	15	0.5213	0.9863	+0.161	
v15 (score_lag1-5, seasonal dev)	—	15	0.1058	1.0470	+0.221	
v17 (score_lag1-7, rolling, XGB-MAE)	—	15	0.0681	1.0815	+0.256	
v23 (ROLL_WINS 90d + LR=0.02 N=2000)	Last-window all	15	—	<i>ausstehend</i>		NB1

Kernbeobachtung: recent_local (8y, Kaggle **0.8095**) = **bestes Modell**. v24 (8y + seasonal_mean, **0.8106**) = **zweitbestes** — knapp dahinter und auf Kaggle stärker als v22 5y (0.8132). **RECENT_YEARS-Sweep (last-window val)**: 5y=0.8132 7y=0.8270(!) 8y=laufend 15y=0.8193 $\rightarrow$ 7y ist ein überraschender Ausreißer nach oben (schlechter), nicht der erwartete Mittelwert. regional_seasonal_mean (Rang 7, 2.9%) hilft: v24 verbessert v22 um 0.0026.

Implementierungsunterschiede

Feature / Design	v12	v15	v16_chain	v17	kaggle_v1	kaggle_v2	kaggle_v3
score_lag1/2/3							
score_lag4-7							
score rolling (mean_4w etc.)							
seasonal deviation							
chain forecast-ing							
wind_roll_* (Noise)							
XGBoost							
MAE-Objective							
X_test = letzter Test-Tag							
X_test = letzter Train-Tag							
LGB + XGB + CatBoost			(LGB only)				
Multi-Seed				(2 Seeds)			
LGB							

Feature Importance (LightGBM Gain)

v19 und v20 — kein score_lag (aktuellste Runs)

Feature	v19	v20 (Schema A)
prec_roll180_mean	16.20 %	16.49 %
prec_roll90_mean	14.02 %	14.27 %
tmp_roll180_max	11.54 %	12.06 %
prec_roll60_mean	8.04 %	8.37 %
regional_mean_score	5.41 %	5.64 %
prec_deficit_90d	3.66 %	3.64 %
prec_roll180_std	2.43 %	2.45 %
wind_roll180_mean	2.35 %	2.35 %
tmp_roll90_max	2.17 %	2.28 %
tmp_roll180_mean	1.71 %	1.64 %

Gruppe	v19	v20
Rolling Stats	79.5 %	79.8 %
Dürre-Indices	7.1 %	6.8 %
regional_mean	5.4 %	5.6 %
Lags	5.3 %	5.2 %
Wetter (direkt)	1.8 %	1.7 %

Rolling Stats dominieren mit ~80 % — lange Zeitfenster (90d, 180d) für Niederschlag und Temperatur sind entscheidend. `prec_roll180_mean` ist konsistent das wichtigste Feature. Kein `score_lag` benötigt.

v21a — Feature-Shift durch längere Rolling-Fenster (270d + 365d)

Gruppe	v19 (180d max)	v21a (365d max)
Rolling Stats	79.5 %	90.6 %
davon roll270	—	16.4 % (NEU)
davon roll365	—	38.2 % (NEU)
Dürre-Indices	7.1 %	1.8 %
regional_mean	5.4 %	1.8 %
Lags	5.3 %	4.0 %
Wetter (direkt)	1.8 %	1.4 %

Top-5 Features v21a: `tmp_roll365_max` (11.5%), `prec_roll365_mean` (10.4%), `prec_roll270_mean` (9.8%), `prec_roll90_mean` (8.4%), `prec_roll60_mean` (6.3%). `prec_roll180_mean` fällt von Rang 1 (16.2%) auf **Rang 7 (4.7%)** — vollständig verdrängt durch noch längere Fenster.

LGB Iterationen: 991/999/**1000**/530/951 — Woche 3 trifft das Limit (N_ESTIMATORS=1000)
→ **Modell war unterfit, brauchte mehr Bäume!**

v22_recent8 — Feature-Shift durch Recent-Filter

Gruppe	v19 (15 Jahre)	v22 (8 Jahre)
Rolling Stats	79.5 %	79.5 %
Dürre-Indices	7.1 %	5.7 %
regional_mean	5.4 %	6.9 %
Lags	5.3 %	4.9 %
Wetter (direkt)	1.8 %	1.7 %

Top-Features Verschiebung: `prec_roll190_mean` wird #1 (war #2), `prec_roll180_mean` fällt auf #2. Das 90-Tage-Fenster gewinnt an Gewicht wenn weniger Langzeitdaten vorhanden sind. `regional_mean_score` steigt von 5.4% auf 6.9% — das Modell kompensiert fehlende History durch die statische Regions-Charakteristik.

Ältere Runs — mit `score_lag`

Feature	v15 (gap=0)	v17 (gap=0)	v16_chain	v18 (gap=13)
<code>score_lag1</code>	78.88 %	76.78 %	91.19 %	41–46 %
<code>score_lag2</code>	12.39 %	—	7.16 %	4–7 %
<code>score_lag3</code>	—	0.73 %	—	1.3 %
<code>score_mean_4w</code>	—	9.34 %	—	—
Alle score-Features	~93 %	93.3 %	~99 %	~50 %
Wetter-Features gesamt	~7 %	~6.7 %	~1 %	~49 %

Mit `gap=13` verschiebt sich die Feature Importance drastisch: `score_lag` fällt von 93% auf 50%, Wetter steigt von 7% auf 49%. Das ist die richtige Richtung — aber selbst 50/50 reicht nicht, weil der stale `score_lag` im Test noch immer schadet. v19 (0% `score_lag`, 100% Wetter, besseres Val-Schema) ist nun das beste Modell mit Kaggle **0.1930**.

Analyse: Warum score_lag schadet

Das Staleness-Problem

Training (Sliding Windows):

X[t] enthält score_lag1 = Score von Woche t → sehr frisch (0 Tage alt)
y[t] = Scores der nächsten 5 Wochen
Autokorrelation lag1 = 0.97 → fast perfekte Vorhersage

Test (Kaggle):

X_test = Features vom letzten Test-Tag (Tag 91 des Test-Fensters)
score_lag1 = letzter TRAININGS-Score (vor 91 Tagen!)
Autokorrelation nach 13 Wochen 0.60–0.70

Ergebnis: Modell gibt 90% Gewicht auf veraltete Info → schlechte Predictions

Szenario	score_lag1 Alter	Autokorr.	Modell-Performance
Validierung (lokal)	1 Woche	~0.97	MAE 0.03–0.10
Kaggle Test	~13 Wochen	~0.60	MAE 1.04–1.08
v12 (kein score_lag)	—	—	MAE 0.82

v18: Gap-Simulation — Ergebnisse & Interpretation

v18 simuliert die 91-Tage-Lücke direkt: score_lag1 = sc[t - 13] in Training **und** Validation.

v18 Ergebnisse:

Val MAE (gap=13, realistisch): 0.1831
Val MAE (gap=0, optimistisch): 0.1275
Kaggle MAE: 0.9134

Persistence-Baseline in Val: 0.0321 ← kritischer Hinweis!
score_lag1 (gap=13) wiederholt: 0.6535

Was gut funktioniert hat: Feature Importance verschob sich von 93% score_lag → 50% score_lag / 49% Wetter. Das Modell nutzt bei stale Scores automatisch mehr Wetter-Signale.

Warum val MAE (0.18) trotzdem Kaggle MAE (0.91):

Die Persistence-Baseline von 0.0321 enthüllt das eigentliche Problem. In den letzten 5 Val-Wochen sind die Dürre-Scores kaum verändert — die Val-Regionen befinden sich am Ende einer stabilen Periode. Der einfachste mögliche Predictor (letzten Score wiederholen) macht MAE 0.03.

Im Kaggle-Test hingegen: echte Unsicherheit über 91 unbekannte Tage. Dürre-Bedingungen können sich in dieser Zeit dramatisch ändern. Das Val-Schema testet die **falsche Frage**.

Val fragt: "Wie stabil ist Dürre in den nächsten 5 Wochen einer bekannten Region?"

Test fragt: "Was passiert in 5 Wochen nach 91 unbekannten Tagen?"

Fazit v18: Gap-Simulation adressiert das score_lag-Staleness-Problem, aber nicht das grundlegende Val-Schema-Problem. Selbst mit gap=13 sind Val-Regionen zu "leicht" weil sie am Ende stabiler Phasen liegen.

Kaggle v18 = 0.9134 > v12 = 0.8258: Selbst gap-trainierter score_lag (50% Gewicht) schadet noch immer im Vergleich zu keinem score_lag. **v19 (0.8193) und v22 (0.8132) sind die besten Modelle.**

v22_recent8: Recent-Filter — Ergebnisse & Interpretation

v22 trainiert auf den letzten 8 von ~15 Jahren pro Region. Gleiche Features und Val-Schema wie v19.

v22_recent8 Ergebnisse:

Val MAE: 0.2342 ← BESSER als v19 (0.2522)
Kaggle MAE: 0.8132 ← BESSER als v19 (0.8193)
Training-Windows: 926.176 (v19: 1.744.448, -47%)
LGB Iterationen: 813/633/608/879/836 (v19: 477/700/388/363/358)
Blend: lgb=0.90 (v19: lgb=0.50, cat=0.45)

Beide Metriken verbessern sich: Val MAE $0.2342 < 0.2522$ und Kaggle MAE $0.8132 < 0.8193$ → kein Overfitting-Muster.

Was erklärt die Verbesserung?

Effekt	Erklärung
Fokus auf relevante Daten	Neuere 8 Jahre haben ähnlichere Muster zum Test-Set

Effekt	Erklärung
Mehr Iterationen = bessere Anpassung	LGB kann mit mehr Bäumen tiefere Muster lernen
Rolling-Features bleiben vollständig	Filter auf Windows, nicht auf Feature-Engineering — lange Rollen bleiben intakt
Val und Kaggle in Einklang	Val MAE und Kaggle MAE verbessern sich gemeinsam → das Val-Schema funktioniert

Was sagen uns die LGB-Iterationen? v22 braucht mehr Bäume (avg ~750 vs 477). Mit weniger Trainings-Windows ist jeder Baum wertvoller — früheres Stoppen war suboptimal bei v19.

Schlussfolgerung: Der Recent-Filter (letzte 8 Jahre) hilft. Die Frage ist: was ist das optimale Fenster?

Merksatz: Wenn Val und Kaggle gemeinsam besser werden → das Daten-Window passt besser zum Test-Set. Der Recent-Filter reduziert Verteilungs-Drift zwischen älteren und neueren Perioden.

RECENT_YEARS Sweep — Vollständige Ergebnisse

Drei Experimente mit unterschiedlichen RECENT_YEARS-Werten zeigen einen klaren Trend:

RECENT_YEARS=5 (33% der Daten):

Kaggle v22 (last-window val, 2248 pts): 0.8132 (NB3)
 local 5y (20%-holdout val, 449 pts): 0.8654 (CSV)

RECENT_YEARS=8 (53% der Daten):

local 8y (20%-holdout val, 449 pts): 0.8095 (CSV) ← BEST

RECENT_YEARS=15 (100% der Daten):

v19 (last-window val, 2248 pts): 0.8193
 v12 (20%-holdout val, 449 pts): 0.8258

Der RECENT_YEARS-Effekt:

RECENT_YEARS	Datenanteil	Kaggle (best. Val-Schema)	Interpretation
5	33%	0.8132	zu wenig Daten → unterfit
8	53%	0.8095	Sweet Spot!
15	100%	0.8193	alte Muster bremsen Generalisierung

Was erklärt den Sweet Spot bei 8 Jahren?

Effekt	Erklärung
Zu wenig Jahre (5y)	LGB trifft Iterations-Limit (alle 1000!) → unterfit, schlechte Generalisierung
Zu viele Jahre (15y)	Klimatische Muster aus 2000–2007 passen nicht mehr zu aktuellen Dürren
8 Jahre = optimaler Mittelweg	Genug Daten für stabiles Training, nah genug an aktuellen Klimamustern

Val-Schema-Effekt bei gleichem RECENT_YEARS=5:

Kaggle v22 (last-window, 2248 val pts): 0.8132
 local 5y (holdout, 449 val pts): 0.8654
 $\Delta = 0.052$ ← großer Unterschied durch besseres Val-Schema!

Größeres Val-Set (2248 pts) → Early Stopping kalibriert besser → Modell konvergiert sauberer. Mit holdout (449 pts) trifft LGB das Iterations-Limit → unkontrolliertes Training → schlechtere Generalisierung.

LGB Iterations-Diagnose:

Setting	LGB Iters (wk1-5)	Problem
local 5y, holdout	1000/1000/998/990/997	Alle am Limit → unterfit
Kaggle v22, 5y, last-win	813/633/608/879/836	Gut kalibriert
local 8y, holdout	215/999/1000/982/998	Woche 2-5 am Limit
Kaggle v22, 15y (v19)	477/700/388/363/358	Gut kalibriert

Kritischer nächster Schritt: recent_local (8y + holdout) = 0.8095, aber Kaggle v22 (5y + last-window) = 0.8132. Wenn wir **8 Jahre + last-window-val** kombinieren → erwartete Verbesserung auf ~0.79–0.80. Die Kombination aus bestem RECENT_YEARS (8y) und bestem Val-Schema (last-window) wurde noch nicht getestet!

v24: 8 Jahre + Seasonal Mean — Ergebnisse & Interpretation

v24 kombiniert RECENT_YEARS=8 mit einem neuen Feature: `regional_seasonal_mean` (Durchschnittsscore pro Region $\times$ Monat, 12 Werte/Region). Sonst identisch mit v19/v22.

v24 Ergebnisse:

```
RECENT_YEARS:      8  (53% der Daten)
Features:          134  (+regional_seasonal_mean)
Val MAE:           0.2314  ← besser als v22/5y (0.2342)
Kaggle MAE:        0.8106  ← 2. bestes Ergebnis insgesamt
LGB iters:         784/987/547/1000/943  ← Woche 4 trifft Limit
Blend:             lgb=0.90 xgb=0.05 cat=0.05
Runtime:           88.4m
```

Was bringt `regional_seasonal_mean`?

Das Feature erreicht Rang 7 (2.88% Gain-Importance) — hinter den dominanten Rolling-Features aber deutlich vor vielen Lags. Es encodiert saisonale Dürremuster pro Region: Gebiete mit ausgeprägten Trockenzeiten werden besser erkannt als durch den jährlichen Gesamtmittelwert allein.

Feature	v19 (15y)	v22 (5y)	v24 (8y + seasonal)
<code>regional_mean_score</code>	5.4%	6.9%	5.3%
<code>regional_seasonal_mean</code>	—	—	2.9% (NEU)
<code>prec_roll90_mean</code>	14.0%	—	16.1% (Rang 1)
<code>prec_roll180_mean</code>	16.2%	—	14.4%

Mit RECENT_YEARS=8 rückt `prec_roll90_mean` auf Platz 1 (war Platz 2 bei 15y). Das kürzere Zeitfenster gewichtet aktuelle Niederschlagsmuster stärker.

Vergleich mit Sweep-Referenz:

Modell	RY	+ seasonal	Kaggle
v22 5y, last-win	5		0.8132
v24 8y, last-win	8		0.8106
recent_local 8y, holdout	8		0.8095

v24 liegt zwischen recent_local (holdout) und v22 (5y, last-window) — der Seasonal-Mean-Effekt ist real, aber klein (~ 0.003). Das Hauptpotenzial liegt weiterhin bei RECENT_YEARS=8 selbst.

LGB Limit bei Woche 4: Iteration 1000 trifft das Limit — Woche 4 war unterfit. Ein erneuter Lauf mit N=1500 oder LR=0.03 könnte minimal verbessern.

v26: RECENT_YEARS=7 — Überraschender Ausreißer

v26 schließt die Lücke im RECENT_YEARS-Sweep zwischen 5y (0.8132) und 8y (laufend). Identische Konfiguration wie v19/v22 (last-window val, 133 Features), nur RECENT_YEARS=7.

v26 Ergebnisse:

```
RECENT_YEARS:    7  (47% der Daten, 809k Train-Fenster)
Val MAE:         0.2273  ← BESTES Val MAE bisher (!)
Kaggle MAE:      0.8270  ← SCHLECHTER als Baseline v12 (0.8258)
LGB iters:       802/1000/706/998/865  ← Wochen 2 und 4 treffen Limit
regional_mean_score: 8.0%  (v24: 5.3%, v19: 5.4%)
Blend:          lgb=0.90 xgb=0.05 cat=0.05
```

Val-MAE 0.2273 = bestes Val, Kaggle 0.8270 = schlechter als Baseline. Paradox.

RECENT_YEARS-Sweep gesamt (last-window val):

```
5y  (v22):  Train ~570k  Val MAE 0.2342  Kaggle 0.8132
7y  (v26):  Train 809k   Val MAE 0.2273  Kaggle 0.8270    Loch!
15y (v19):  Train 1,744k Val MAE 0.2522  Kaggle 0.8193
```

7y ist der schlechteste der drei last-window Punkte — obwohl die Val-MAE optimal ist.

Mögliche Erklärungen:

Ursache	Erklärung
LGB unterfit (2 Wochen Limit)	Wochen 2 und 4 nie konvergiert → schlechte Predictions für diese Wochen
7y trifft klimatische Ausreißerperiode	Das 7-Jahres-Fenster enthält zufällig eine besonders variable Periode

Ursache	Erklärung
Val MAE-Verbesserung durch Overfitting	Modell lernt Val-Muster auswendig, überträgt sich nicht auf Test
<code>regional_mean_score</code> springt auf 8%	Modell stützt sich stark auf statischen Mittelwert statt Zeitreihen → weniger robust

Kern-Diagnose: Die zwei LGB-Limit-Treffer bei Wochen 2 und 4 sind der wahrscheinlichste Schuldige. Das Modell konvergiert für diese Wochen nicht → Predictions für Woche 2 und 4 sind systematisch schlechter, was den Kaggle-MAE über alle 5 Wochen stark anhebt.

Schlussfolgerung: 7y mit N=1000 ist unterfit. Entweder N=2000 testen (wie v21b-Ansatz) oder 7y überspringen und stattdessen 8y bestätigen. Der RECENT_YEARS-Sweet-Spot ist **8y, nicht 7y**.

v21a: Längere Rolling-Fenster (270d + 365d) — Ergebnisse & Interpretation

v21a fügt 270-Tage- und 365-Tage-Rolling-Fenster hinzu. Gleiche Val-Schema und Features wie v19 (außer ROLL_WINS).

v21a Ergebnisse:

```

Val MAE:          0.2272  ← BESSER als v19 (0.2522) und v22 (0.2342)
Kaggle MAE:       0.8671  ← SCHLECHTER als v12 (0.8258), v19 (0.8193), v22 (0.8132)
Features:         157    (v19: 133, +24 neue Rolling-Features)
LGB Iterationen:  991/999/1000/530/951  ← Woche 3 trifft Limit!
Blend:            lgb=0.90, xgb=0.05, cat=0.05
Runtime:          178.4m

```

Gleiche Warnung wie v22: Val verbessert sich, Kaggle verschlechtert sich.

Warum scheitert v21a trotz besserer Val-MAE?

Effekt	Erklärung
91-Tage-Grenze im Test-Set	Test hat nur 91 Tage pro Region — roll270/365 werden wie roll91 berechnet
Train/Test-Distributions-Shift	Im Training: roll365 = echter Jahres-Mittelwert. Im Test: roll365 = 91-Tage-Mittelwert

Effekt	Erklärung
Val sieht volle History	Val-Punkte kommen aus Trainingsdaten (15 Jahre vorhanden) → roll365 ist korrekt
Val und Test messen verschiedene Dinge	Val-Feature Test-Feature für roll270/365
Blend-Kollaps	LGB=0.90 wieder — Val-leicht-Features lassen LGB dominieren

Die 91-Tage-Grenze ist der entscheidende Befund:

Trainings-Daten: roll365_mean = Durchschnitt der letzten 365 Tage sinnvoll
Test-Daten (91d): roll365_mean = Durchschnitt der letzten 91 Tage irreführend!

Das Modell lernt: "roll365_mean ist das wichtigste Feature (10.4%)"
Aber im Test gibt es kein echtes 365-Tage-Fenster → Feature ist ein anderes Konzept
→ Das gelernte Muster überträgt sich nicht

LGB Iterationen als Signal: Woche 3 trifft genau N_ESTIMATORS=1000 (nie konvergiert). Das bedeutet: **v19 und v21a waren unterfit** — 1000 Bäume reichen nicht, mehr wären besser.

Woche	v21a Iters	v19 Iters	Δ
1	991	477	+514
2	999	700	+299
3	1000 (Limit!)	388	+612
4	530	363	+167
5	951	358	+593

Schlussfolgerung: Längere Rolling-Fenster (>180d) sind kontraindiziert wenn das Test-Set 91 Tage hat. Die optimale maximale Fenstergröße ist durch die Test-Datenlänge begrenzt (~90 Tage = max sinnvoll). Die v19-Roll-Fenster (max 180d) sind bereits "zu lang" für den Test — aber 180d funktioniert noch weil das Modell mit min_periods=18 zumindest einen Teil des echten Signals bekommt.

Nächster Schritt: v21b — mehr Bäume (N=2000, LR=0.02) bei gleichen Features wie v19. Damit könnte das Konvergenzproblem (Woche 3 trifft Limit) behoben werden.

Merksatz: Rolling-Fenster müssen zur Test-Datenlänge passen. Features die im Test nicht korrekt berechnet werden können, schaden der Generalisierung — egal wie gut sie im Val sind.

v21b: Mehr Bäume (LR=0.02, N=2000, ×1.1) — Ergebnisse & Interpretation

v21b adressiert das Konvergenzproblem aus v21a: LR 0.04→0.02 und N_ESTIMATORS 1000→2000 damit LGB nicht mehr am Limit stoppt. Gleiche Features und Val-Schema wie v19.

v21b Ergebnisse:

Val MAE: 0.2528 + leicht schlechter als v19 (0.2522)
Kaggle MAE: 0.8192 + praktisch identisch mit v19 (0.8193)
Runtime: 251m + 2× länger als v19 (130m)
LGB Iterationen: 396/1168/1077/1022/1394 (avg 1011)
Blend: lgb=0.50 xgb=0.05 cat=0.45 (identisch mit v19)
Final trees (×1.1): [435, 1284, 1184, 1124, 1533]

Das Modell brauchte tatsächlich mehr Bäume — aber es half nicht.

Woche	v19 Iters (LR=0.04)	v21b Iters (LR=0.02)	Faktor
1	477	396	0.8×
2	700	1168	1.7×
3	388	1077	2.8×
4	363	1022	2.8×
5	358	1394	3.9×

Wochen 2–5 brauchten deutlich mehr Bäume mit LR=0.02 — sie haben jetzt sauber konvergiert statt am Limit zu stoppen. Trotzdem: Kaggle-Score bleibt 0.8192 0.8193.

Warum hilft mehr Konvergenz nicht?

Hypothese	Bewertung
v19 war nicht unterfit, sondern hatte echte Kapazitätsgrenze	Wahrscheinlich — mehr Bäume = mehr Overfitting ohne neues Signal
Der Bottleneck liegt bei den Features, nicht bei der Modellkapazität	Feature-Set (133 Features) bleibt identisch
Val-Schema (last-window, 2248 pts) limitiert die Optimierung stärker als N_ESTIMATORS	Blend-Gewichte bleiben gleich → identische Modell-Struktur
×1.1 Final-Scaling war zu konservativ	Möglich, aber kleine Änderung

Schlussfolgerung: Der Engpass liegt nicht bei der Baumanzahl, sondern bei Features oder RECENT_YEARS. v21b bestätigt v19 als solides Baseline. Verbesserungen kommen durch

andere Datenauswahl (RECENT_YEARS=8) oder **andere Features** (seasonal mean, kürzere Rolls).

v25: Last-Year Validation (52 Fenster/Region) — Ergebnisse & Interpretation

v25 testet ein breiteres Val-Schema: statt 1 Fenster pro Region werden die letzten 52 Fenster (~1 Jahr) als Val genutzt. RECENT_YEARS=9, sonst identisch mit v22.

v25 Ergebnisse:

RECENT_YEARS: 9 (~60% der Daten)
 Val-Schema: letzte 52 Fenster pro Region (~116.896 Punkte)
 Train-Fenster: erste ~8 Jahre pro Region
 Kaggle MAE: 0.8163 ← schlechter als best (0.8095), besser als v19 (0.8193)
 Val MAE: nicht direkt vergleichbar mit v22 (anderes Schema)

Vergleich — gleiche effektive Trainingsdatenmenge:

Setting	Training auf	Val-Schema	Val-Punkte	Kaggle
recent_local 8y (beste)	8 Jahre	20%-Holdout (449 Regionen)	449	0.8095
v25 (9y last-year)	~8 Jahre (9y – 1y val)	52 Fenster/Region	~116.896	0.8163

Effektiv trainieren beide Modelle auf denselben ~8 Jahren Daten — der Unterschied liegt nur im Val-Schema. Das Holdout (449 Punkte über 449 Regionen) liefert bessere Early-Stopping-Kalibrierung als 52 Fenster pro Region.

Warum hilft mehr Val nicht?

Hypothese	Erklärung
Holdout = Regions-Generalisierung	449 Regionen testen ob Modell auf neue Regionen überträgt → näher am Kaggle-Test
52 Fenster/Region = Zeit-Generalisierung	Testet ob Modell in der Zeit extrapoliert → andere Frage als Kaggle stellt
116k Val-Punkte → Early Stopping zu sensitiv	Modell stoppt zu früh weil Val-Signal over-averaged ist
Training verliert 1 komplettes Jahr	Bei 9y – 1y = 8y Trainingsdaten kein Vorteil über direkte 8y

Schlussfolgerung: Das Last-Year Val-Schema ist trotz $52\times$ mehr Validierungspunkten nicht besser als Holdout oder Last-Window. Der Kaggle-Test prüft Regions-Generalisierung stärker als Zeitgeneralisierung. **Holdout-Val bleibt das robusteste Schema für Early Stopping.**

Validierungsschema – Analyse & Alternativen

Aktuelles Schema

Region-Holdout (20%):

Train: 80% der Regionen, alle Windows (stride=1)

Val: 20% der Regionen, NUR letztes Window pro Region

→ 449 Datenpunkte ($2248 \times 20\%$)

→ score_lag1 = Score von vor 1 Woche (immer frisch!)

Problem: Das Val-Schema testet die falsche Schwierigkeitsstufe. Im Training/Val ist score_lag immer frisch (Autokorr. 0.97). Im Kaggle-Test ist score_lag 13 Wochen alt (Autokorr. 0.60–0.70). Das führt zur riesigen Diskrepanz: Val-MAE 0.03 vs. Kaggle-MAE 0.82.

Unbekannte Teststruktur

Wir wissen nicht genau, wie der Kaggle-Test aufgebaut ist:

Hypothese	Beschreibung	Konsequenz für score_lag
A (<i>wahrscheinlich</i>)	Gleiche Regionen, Prediction nach 91-Tage-Fenster	score_lag = 13+ Wochen alt → veraltet
B	Gleiche Regionen, Prediction innerhalb des 91-Tage-Fensters	score_lag = 1-13 Wochen alt → teils nützlich
C	Völlig neue Regionen (nicht im Training)	score_lag aus Training komplett nutzlos

v12's Überlegenheit deutet auf **Hypothese A oder C** hin: Wetter-Features sind robuster als score_lag.

Alternative Validierungsstrategien

Option 1: Time-based Split (empfohlen als Vergleich)

```
# Statt Region-Holdout: für ALLE Regionen letzte N Wochen als Val
for region, g in weekly.groupby("region_id"):
    train_part = g.iloc[:-N] # alles bis auf letzte N Wochen
    val_part    = g.iloc[-N:] # letzte N Wochen
```

Vorteile: - Jede Region hat einen Val-Punkt → ~2248 Datenpunkte statt 449 - Testet Zeit-generalisierung innerhalb bekannter Regionen

Nachteile: - score_lag noch immer frisch (kein simulierter 91-Tage-Gap)

Option 2: Gap-Simulation (realistischstes Val-Schema)

```
# Für jede Region: score_lag1 = Score von VOR 13 Wochen (simuliert 91-Tage-Gap)
# X[t] = Features bei Woche t
# score_lag1_test = sc[t - 13] (statt sc[t])
# y = Scores bei Woche t+1..t+5
```

Vorteile: - Simuliert exakt das Testszenario (score_lag 13 Wochen alt) - Val-MAE würde realistisch Kaggle-MAE werden - Würde zeigen ob score_lag mit GAP noch nützlich ist

Nachteile: - Komplexere Implementierung - Reduziert Trainings-Datenpunkte pro Region

Option 3: Gemischte Strategie

Train: alle Windows, score_lag immer frisch

Val: wie Testszenario, score_lag = 13 Wochen alt (Gap-Simulation)

Resultat: Modell lernt mit frischem score_lag zu trainieren, wird aber an realem Testszenario evaluiert. Der Kaggle-Score würde in val besser vorhergesagt.

Empfehlung: Was testen?

Erkenntnisstand: score_lag schadet in allen Varianten. v12 (kein score_lag) ist bestes Modell. **Strategie:** Wetter-Features verbessern, nicht score_lag.

Test	Priorität	Erwarteter Erkenntnisgewinn
kaggle_v1_base (kein score_lag)		Reproduziert v12 mit mehr Features — kann es besser sein?
Time-based Val (lokal)		Testet ob Val-MAE realistischer wird (letzter N Wochen aller Regionen)
RECENT_YEARS = 3 oder 5		Fokus auf aktuellste Muster, weniger historischer Drift
kaggle_v2_seasonal		Isoliert seasonal dev — hilft es ohne score_lag?
XGBoost-only (kein LGB)		XGB hatte 95% Blend-Gewicht, LGB scheint schwächer
kaggle_v3_scorlag		Weniger sinnvoll — score_lag schadet laut v18 auch mit gap=13

Nächste Schritte

Laufend: kaggle_v1_base auf Kaggle

Lokal parallel testen (ein Script):

Option A — Time-based Val: letzten 8–13 Wochen aller Regionen als Val verwenden. Wenn Val-MAE dann 0.82 → Schema-Problem bestätigt, können val als Optimierungsmetrik nutzen.

Option B — RECENT_YEARS: v1_base mit RECENT_YEARS=5 laufen lassen. Wenn Kaggle-Score besser → ältere Daten schaden (anderen Klimaperioden).

Nächste Submission-Runde (nach v1_base Ergebnis): - Ist v1 > v12 → Basis ist korrekt, verbessern mit time-based val + feature-Tuning - Ist v1 < v12 → v1 hat bessere Wetter-Features → darauf aufbauen - Ist v1 > v12 → v12 Features waren spezifisch besser → v12 analysieren

Technische Details

Datenstruktur NPZ

```
# train.npz
# Keys: region_id (int32), region_names (str), year, month, day (int32)
#       prec, surf_pre, humidity, tmp, dp_tmp, wb_tmp,
#       tmp_max, tmp_min, tmp_range, surf_tmp,
#       wind, wind_max, wind_min, wind_range (float32)
#       score (float32)

# test.npz: identisch, ohne 'score'

# Ordinal: year*372 + month*31 + day (proprietäres Kalender-System)
```

Cache-Strategie

cache_weekly.npz	(~495 MB)	Feature Engineering, geteilt v1/v2/v3
cache_windows_v1.npz	(~855 MB)	Sliding Windows für v1
cache_windows_v2.npz		Sliding Windows für v2
cache_windows_v3.npz		Sliding Windows für v3

Modell-Parameter (alle Scripts)

```
LGB_PARAMS = dict(
    n_estimators=1000, learning_rate=0.04, num_leaves=127,
    min_child_samples=60, subsample=0.8, colsample_bytree=0.8,
    reg_alpha=0.1, reg_lambda=0.1
)
XGB_PARAMS = dict(
    n_estimators=1000, learning_rate=0.04, max_depth=6,
    min_child_weight=50, subsample=0.8, colsample_bytree=0.8
)
# Blend-Gewichte: automatisch optimiert auf Val-MAE (Grid-Suche, 5%-Schritte)
```